

Analisi e Progettazione del Software

Diagrammi di sequenza di sistema

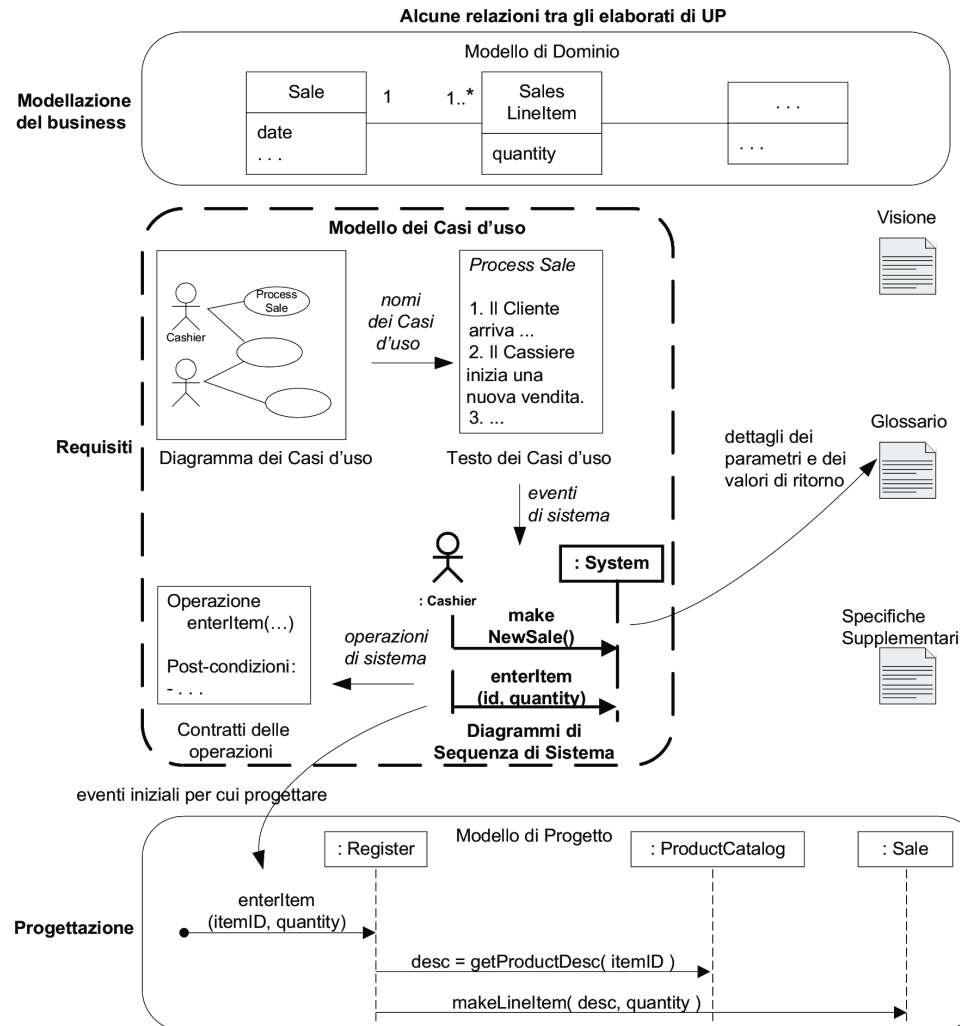
Oliviero Riganelli

Introduzione

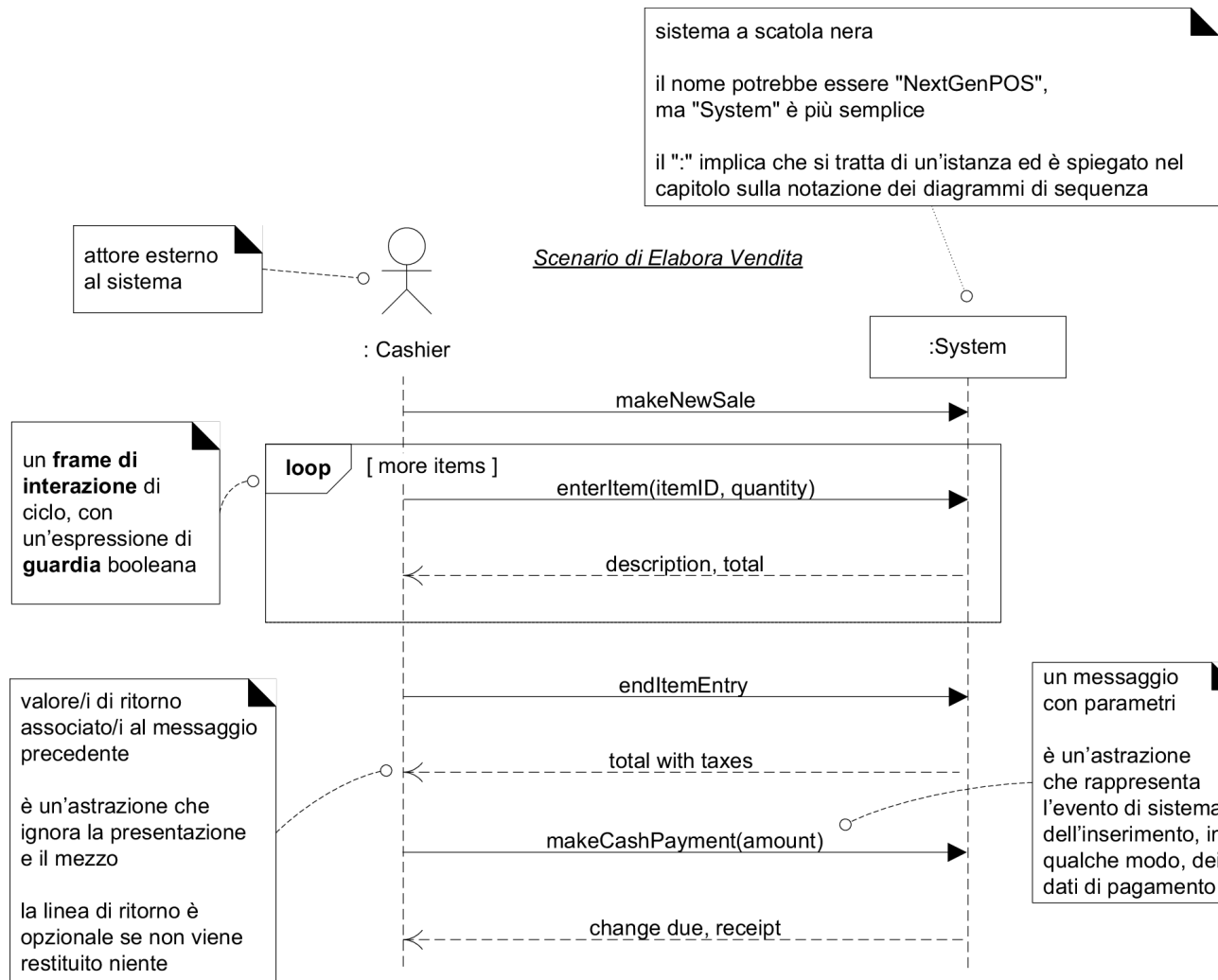
Un diagramma di sequenza di sistema (SSD) mostra gli eventi di input e output dei sistemi in discussione

- Per un particolare corso di eventi all'interno di un caso d'uso mostra:
 - *Gli attori esterni che interagiscono direttamente con il sistema*
 - *Il sistema (a scatola nera)*
 - *Gli eventi di sistema generati dagli attori*

Influenza tra alcuni elaborati di UP



Esempio: NextGen



Che cosa sono gli eventi e le operazioni di sistema

Casi d'uso descrivono il modo in cui gli attori esterni interagiscono con il sistema

- Un attore genera degli **eventi di sistema** per richiedere l'esecuzione di alcune **operazioni sistema**
 - *Le operazioni sistema sono operazioni che il sistema deve definire per gestire gli eventi di sistema*

Che cosa sono gli eventi e le operazioni di sistema

- In UML un **evento** è qualcosa di importante o degno di nota che avviene durante l'esecuzione di un sistema
 - Un *evento di sistema* è un evento esterno al sistema, di input, di solito generato da un attore per interagire con il sistema
- In UML un'**operazione** rappresenta una trasformazione oppure un'interrogazione che un oggetto o componente può essere chiamato eseguire
 - Un'*operazione di sistema* è una trasformazione oppure un'interrogazione che il sistema può essere chiamato a eseguire

Che cosa sono i diagrammi di sequenza di sistema

- UML fornisce la notazione dei **diagrammi sequenza** per illustrare interazioni tra attori e le operazioni iniziate da essi
 - Un **diagramma di sequenza di sistema** è una diagramma che mostra, per un particolare scenario di un caso d'uso, gli eventi generati dagli attori esterni al sistema, il loro ordine e gli eventi inter-sistema (ovvero tra sistemi)

Linea guida

Disegnare un SSD per uno scenario principale di successo di ciascun caso chiuso, nonché per gli scenari alternativi più frequenti o complessi

Applicare UML: Eventi, operazioni e SSD

- UML non definisce ne *eventi di sistema* o *operazioni di sistema* ne *diagrammi di sequenza di sistema*
- UML definisce degli elementi o diagrammi chiamati semplicemente *evento*, *operazione* e *diagramma di sequenza*
- La qualifica *di sistema* enfatizza l'applicazione di questo tipo di elementi o diagrammi ai sistemi, considerati a scatola nera

Perché disegnare un SSD

Gli eventi di sistema sono importanti

- Il **software** deve essere **progettato per gestire** gli **eventi di sistema**

Un sistema software reagisce a tre cose

- **eventi esterni** da parte di attori (umani o sistemi informatici)
- **eventi temporali**
- **Guasti o eccezioni** (spesso provenienti da sorgenti esterne)

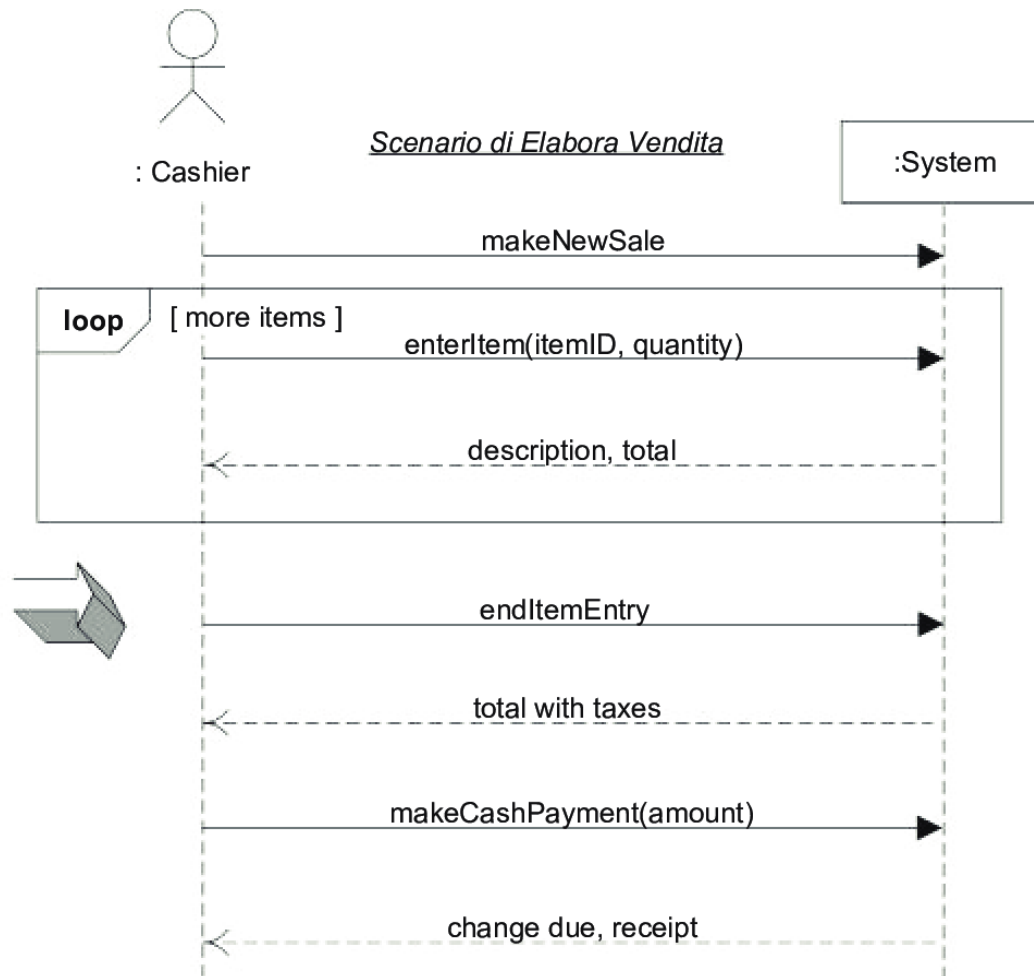
Come descrivere le funzioni e il comportamento a “**scatola nera**” del sistema?

- Descrivere **che cosa** fa il sistema, senza spiegare *come* lo fa
- modelli
 - *casi d'uso*
 - *diagrammi di sequenza di sistema*
 - *contratti delle operazioni di sistema*

Relazione tra SSD e casi d'uso

Scenario di base di *Elabora Vendita*, con pagamento in contanti:

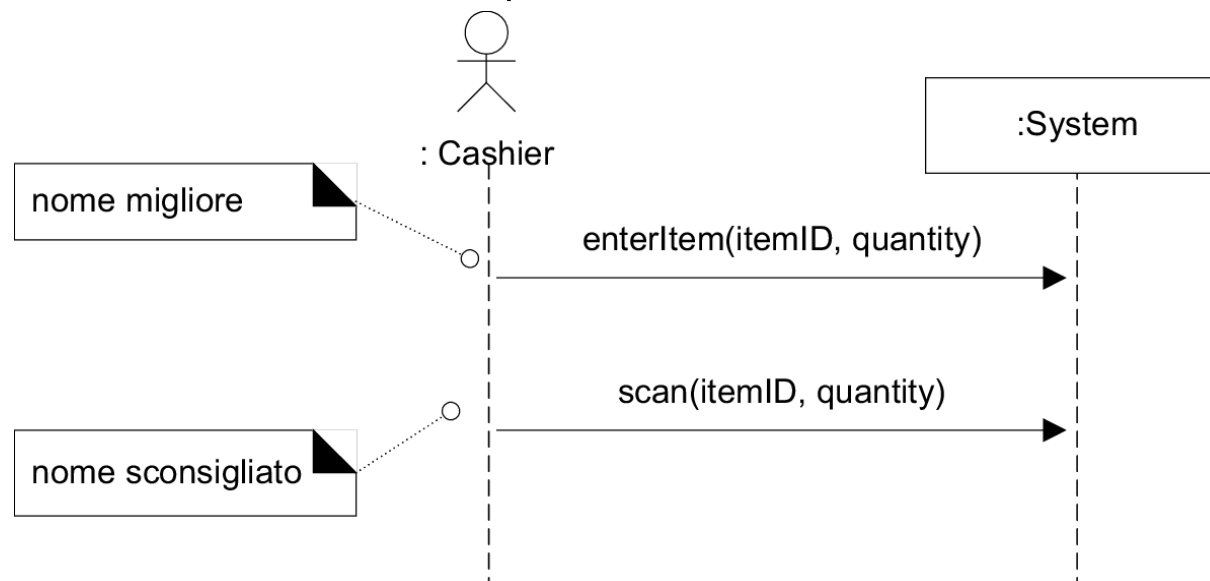
1. Il Cliente arriva alla cassa POS con gli articoli e/o i servizi da acquistare.
2. Il Cassiere inizia una nuova vendita.
3. Il Cassiere inserisce il codice identificativo di un articolo.
4. Il Sistema registra la riga di vendita per l'articolo e mostra una descrizione dell'articolo, il suo prezzo e il totale parziale.
Il Cassiere ripete i passi 3-4 fino a che non indica che ha terminato.
5. Il Sistema mostra il totale.
6. Il Cassiere riferisce il totale al Cliente, e richiede il pagamento.
7. Il Cliente paga (in contanti) e il sistema gestisce il pagamento.
8. Il Sistema registra la vendita completata.
9. Il Sistema genera la ricevuta.
10. Il Cliente va via con la ricevuta e gli articoli acquistati.



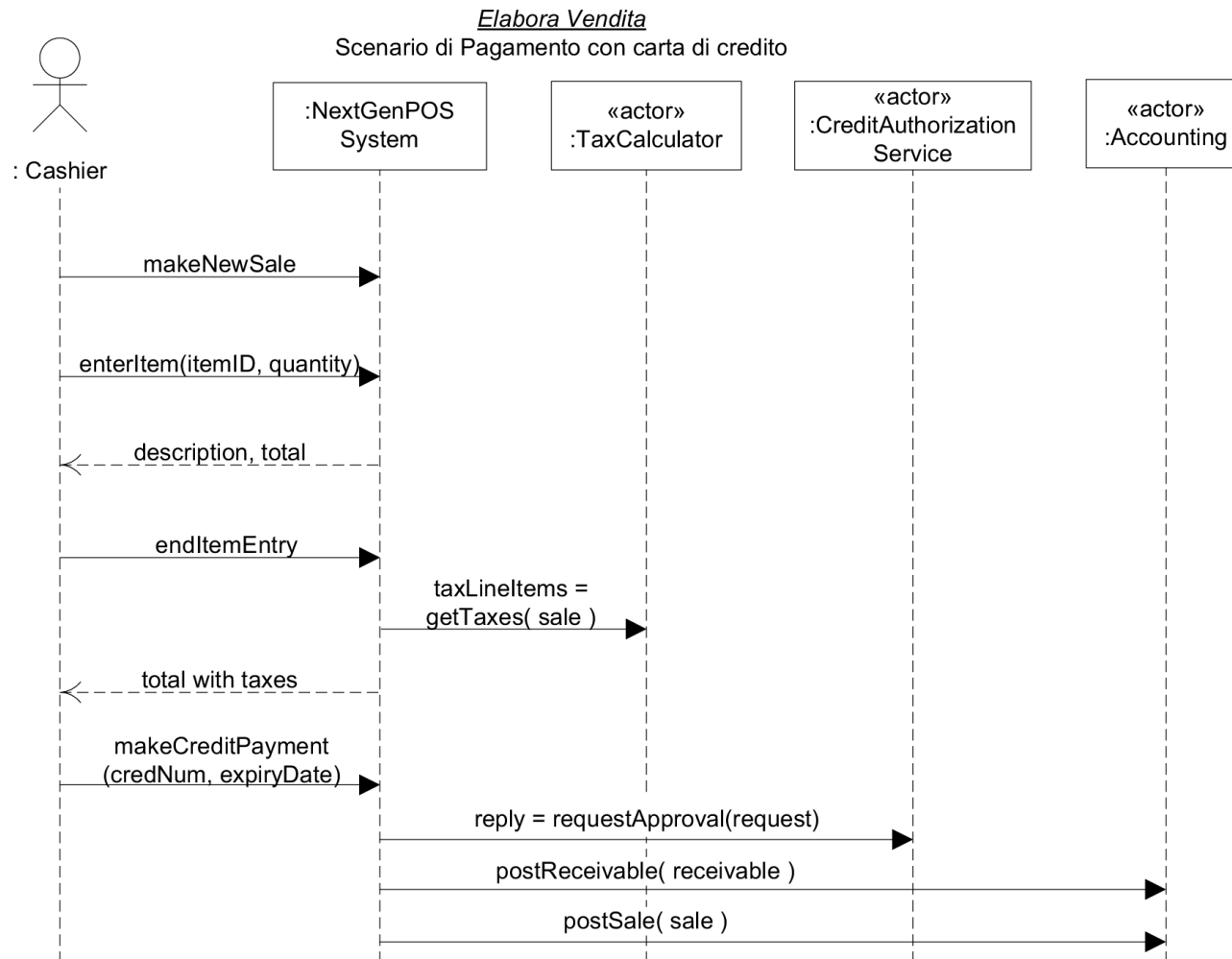
Assegnare il nome a eventi e operazioni di sistema

Eventi di sistema e operazioni corrispondenti

- Devono essere espressi a un livello astratto, **intenzione**, anziché azione.
- Iniziare il nome con un verbo (add..., enter..., end..., make...)



Modellare SSD che coinvolgono altri sistemi esterni



SSD e Glossario

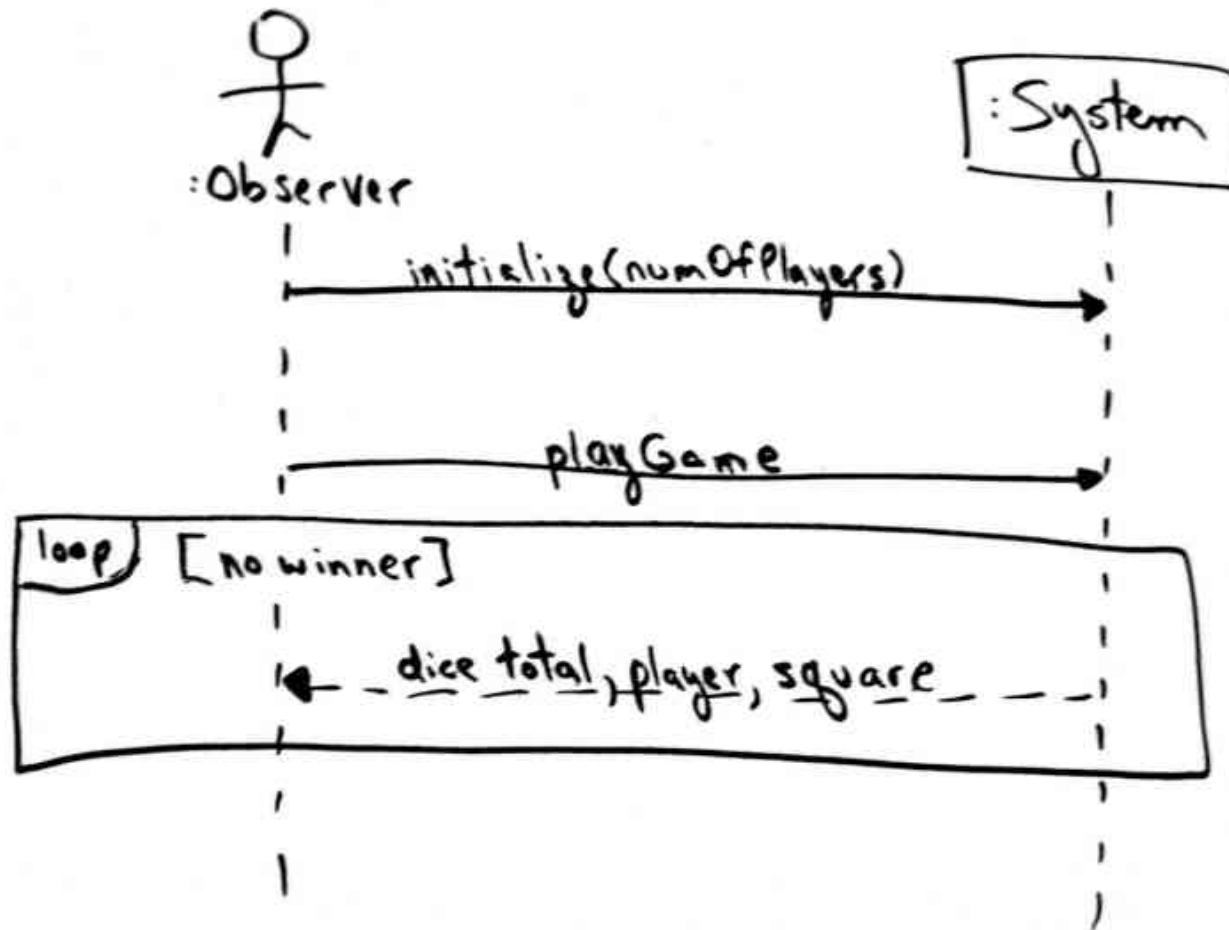
Gli elementi mostrati negli SSD (operazioni, parametri, valori restituiti) sono concisi

- Probabilmente è necessaria una spiegazione più appropriata di questi elementi, in modo che durante la progettazione risulti chiaro cosa entra e cosa esce
- il Glossario può descrivere questi termini in maggior dettaglio

Linea guida

In generale, è opportuno mostrare nel Glossario i dettagli di molti elaborati

Monopoly: SSD per uno scenario di Gioca una Partita



SSD evolutivi e iterativi

Creazione di SSD

- scopo: comprendere
- non per tutti gli scenari
- solo per quelli dell'iterazione corrente (per ogni scenario principale di successo e per gli scenari alternativi più frequenti o complessi)

Gli SSD sono parte del Modello dei casi d'uso

- visualizzano interazioni implicate dai casi d'uso
- maggiormente creati durante l'elaborazione
- non sono un elaborato ufficiale di UP, anche se i creatori di UP ne riconoscono l'utilità

Analisi e Progettazione del Software

Contratti delle operazioni di sistema

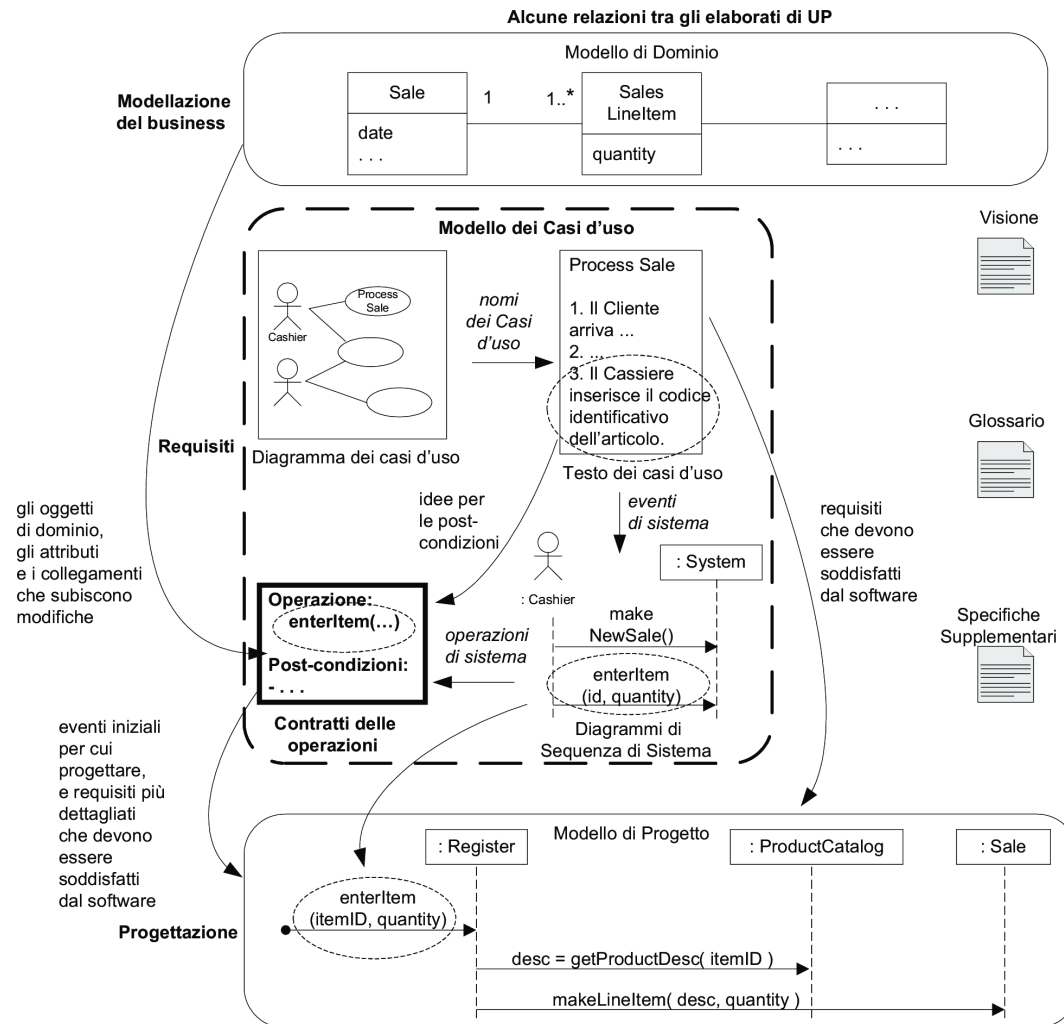
Oliviero Riganelli

Introduzione

Il comportamento del sistema è descritto dai casi d'uso e dai requisiti funzionali

- i contratti delle operazioni di sistema permettono di descrivere il comportamento del sistema in modo più dettagliato
 - *usano pre-condizioni e post-condizioni per descrivere i cambiamenti agli oggetti in un modello di dominio*

Influenze tra alcuni elaborati UP



Esempio

Contratto CO2: enterItem

Operazione:	enterItem(itemID: ItemID, quantity: integer)
--------------------	--

Riferimenti:	casi d'uso: Elabora Vendita
---------------------	-----------------------------

Pre-condizioni:	è in corso una vendita s
------------------------	--------------------------

Post-condizioni:	- è stata creata un'istanza sli di SalesLineItem (<i>creazione di oggetto</i>).
-------------------------	---

	- sli è stata associata con la Sale (vendita) corrente s (<i>formazione di collegamento</i>).
--	---

	- sli è stata associata con una ProductDescription, in base alla corrispondenza con itemID (<i>formazione di collegamento</i>).
--	---

	- sli.quantity è diventata quantity (<i>modifica di attributo</i>).
--	---

Le sezioni di un contratto

Operazione:	Nome e parametri (firma) dell'operazione.
Riferimenti:	Casi d'uso in cui può verificarsi questa operazione.
Pre-condizioni:	Ipotesi significative sullo stato del sistema o degli oggetti nel Modello di Dominio prima dell'esecuzione dell'operazione. Si tratta di ipotesi non banali, che dovrebbero essere comunicate al lettore.
Post-condizioni:	È la sezione più importante. Descrive i cambiamenti di stato degli oggetti nel Modello di Dominio dopo il completamento dell'operazione.

Che cosa è un operazione di sistema

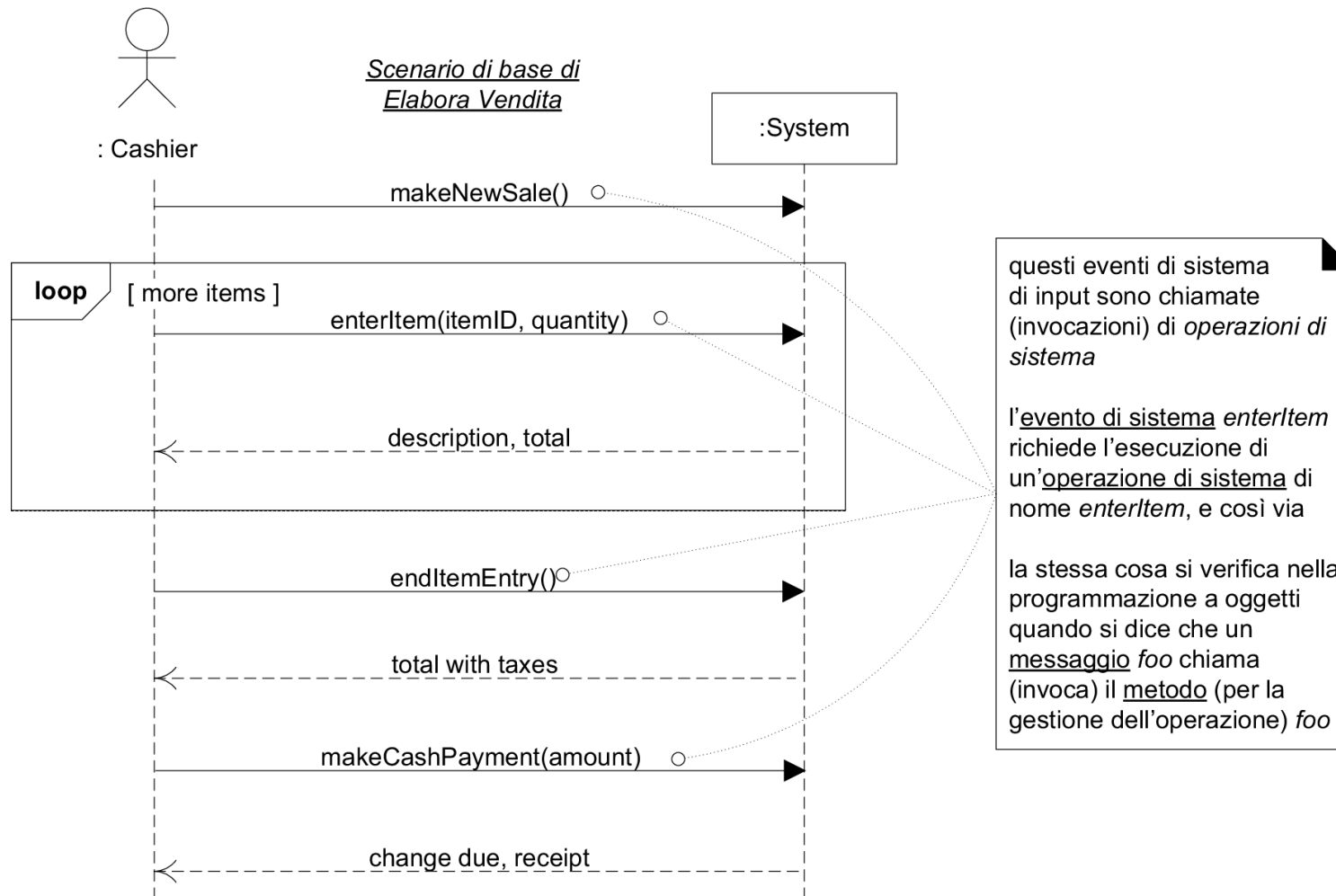
Una **operazione di sistema** è una operazione pubblica del sistema

- l'esecuzione è richiesta da un evento di sistema
- l'esecuzione di una operazione di sistema cambia lo stato del sistema – rappresentato dallo stato degli oggetti nel modello del domino
- interfaccia del sistema – l'insieme di tutte operazioni di sistema

Un **contratto**

- relativo a una operazione di sistema
- descrive il cambiamento dello stato di oggetti del Modello del dominio causato dall'esecuzione dell'operazione di sistema

Le operazioni di sistema gestiscono gli eventi di sistema di input



Post-condizioni

- descrivono cambiamenti nello stato degli oggetti del Modello di dominio
- non descrivono azioni eseguite durante l'esecuzione
 - *sono dichiarazioni circa lo stato degli oggetti dopo l'esecuzione dell'operazione*
 - *scritte al passato*
- possibili cambiamenti di stato
 - *creazione o cancellazione di oggetto (istanza di una classe)*
 - *cambiamento di valore di attributo*
 - *formazione o rottura di collegamento (istanza di un'associazione)*
- perché importanti?
 - *descrivono i cambiamenti richiesti dall'esecuzione dell'operazione di sistema, senza però descrivere come ottenere questi cambiamenti*

Lo spirito delle post-condizioni: il palcoscenico e il sipario

Il sistema e i suoi oggetti sono mostrati sul palcoscenico di un teatro

- Prima dell'operazione, si fa una foto al palcoscenico
- Si chiude il sipario sul palcoscenico e si applica l'operazione di sistema (rumore sul palco)
- Si apre il sipario e si fa una seconda foto
- Si confrontano le foto prima e dopo: le post-condizioni vanno espresse come cambiamenti di stato sul palcoscenico (**è stata** creata una SalesLineItem ...)

Linea guida

Le post-condizioni vanno espresse con un verbo al passato

Esempio: post condizioni per enterItem

Creazione o cancellazione di oggetti

- Dopo che il cassiere ha inserito *ItemID* e *quantity* di un articolo quali nuovi oggetti devono essere creati?
 - È stata creata una istanza *sli* di *SalesLineItem*

Formazione o rottura di collegamenti

- Dopo che il cassiere ha inserito *ItemID* e *quantity* di un articolo quali collegamenti tra oggetti devono essere formati o spezzati?
 - *sli* è stata associata con la *Sale* (vendita) corrente *s* (formazione di collegamento)
 - *sli* è stata associata con una *ProductDescription*, in base alla corrispondenza con *ItemID*

Modifica di attributi

- Dopo che il cassiere ha inserito *ItemID* e *quantity* di un articolo quali attributi di oggetti devono essere modificati?
 - *sli.quantity* è diventata *quantity* (modifica di attributo)

Esempio: pre-condizioni per *enterItem*

Le precondizioni descrivono ipotesi significative sullo stato del sistema prima dell'esecuzione all'operazione (**descrizione sintetica dello stato di avanzamento del caso d'uso**)

- L'operazione *enterItem* viene eseguita quando
 - una vendita è già iniziata ma non è ancora conclusa
 - È stata eseguita l'operazione *makeNewSale* (esattamente una volta)
 - L'operazione *enterItem* potrebbe essere già stata eseguita (zero o più volte)
- Lo stato di avanzamento del caso di uso si può riassumere con la precondizione “è incorso una vendita”

Adottare un punto di vista concettuale e della conoscenza

Il cambiamento dello stato del sistema causato dall'esecuzione dell'operazione di sistema va espresso in termini di oggetti, collegamenti e attributi nel dominio di interesse che il sistema ha necessità di *conoscere e ricordare*

- “È stata creata un'istanza X” va interpretata come “è iniziata la conoscenza da parte del sistema dell'istanza X”
- Nell'operazione *enterItem*, l'inizio della conoscenza dell'istanza di *SalesLineItem* da parte del sistema coincide con l'inizio della sua esistenza
- Non è lecito scrivere una post-condizione “è stata creata un'istanza di *ProductDescription*” tra le post-condizioni di *enterItem*
 - *Il sistema già conosce tutte le descrizioni dei prodotti in vendita nel negozio*

Aggiornamento del modello di dominio

- Durante la creazione dei contratti è normale che emerga la necessita di registrare nuove classi concettuali, attributi o associazioni nel modello di dominio

Linea guida

Nei metodi iterativi e evolutivi, tutti gli elaborati dell'analisi e della progettazione sono considerati parziali e imperfetti, ed evolvono in risposta a nuove scoperte

Utilità dei contratti

Requisiti funzionali e comportamento del sistema in UP

- Principalmente, casi d'uso
 - *i contratti non sono sempre necessari*
- i contratti consentono di rappresentare situazioni dettagliate o complesse che non è opportuno descrivere nei casi d'uso
 - *i contratti sono complementari ai casi d'uso*

Come creare e scrivere i contratti

1. Identificare le operazioni di sistema dagli SSD
 2. Creare i contratti per le operazioni più complesse, o che non sono chiare dai caso d'uso
 3. Per descrivere le post-condizioni utilizzare le seguenti categorie
 - *creazione o cancellazione di oggetto*
 - *Modifica di attributo*
 - *formazione o rottura di collegamento*
- Le post-condizioni descrivono cambiamenti di stato al passato
 - ricordare di formare collegamenti necessari tra oggetti esistenti e quelli appena creati
 - le pre-condizioni sono normalmente implicate dall'esecuzione di operazioni di sistema “precedenti”
 - *utili per indicare quali oggetti sono noti al sistema*

Contratti per POS NextGen

Contratto CO1: makeNewSale

Operazione: makeNewSale()

Riferimenti: casi d'uso: Elabora Vendita

Pre-condizioni: nessuna

Post-condizioni:

- è stata creata un'istanza s di Sale (*creazione di oggetto*).
- s è stata associata con il Register (*formazione di collegamento*).
- gli attributi di s sono stati inizializzati (*modifica di attributi*).

Contratti per POS NextGen

Contratto CO2: enterItem

Operazione: enterItem(itemID: ItemID, quantity: integer)

Riferimenti: casi d'uso: Elabora Vendita

Pre-condizioni: è in corso una vendita s.

Post-condizioni:

- è stata creata un'istanza sli di SalesLineItem (*creazione di oggetto*).
- sli è stata associata con la Sale (vendita) corrente s (*formazione di collegamento*).
- sli è stata associata con una ProductDescription, in base alla corrispondenza con itemID (*formazione di collegamento*).
- sli.quantity è diventata quantity (*modifica di attributo*).

Contratti per POS NextGen

Contratto CO3: endItemEntry

Operazione:	endItemEntry()
Riferimenti:	casi d'uso: Elabora Vendita
Pre-condizioni:	è in corso una vendita s.
Post-condizioni:	- nessuna.

E' meglio evitare di scrivere contratti per operazioni che sono solo interrogazioni

Contratti per POS NextGen

Contratto CO4: makeCashPayment

Operazione: makeCashPayment(amount: Money)

Riferimenti: casi d'uso: Elabora Vendita

Pre-condizioni: è in corso una vendita s.

Post-condizioni:

- è stata creata un'istanza p di CashPayment (*creazione di oggetto*).
- p è stata associato con la Sale (vendita) corrente s (*formazione di collegamento*).
- p.amountTendered è diventato amount (*modifica di attributo*).
- la Sale corrente s è stata associata con lo Store (*formazione di collegamento*); (s è stata aggiunta al registro storico delle vendite completate).

Applicare UML: operazioni, contratti e OCL

In UML

- una **operazione** è la specifica di una trasformazione o interrogazione che un oggetto può essere chiamato ad eseguire
- un **metodo** è l'implementazione di una operazione
- una operazione ha una **firma (signature)** e è associata a un insieme di **vincoli (Constraint)** classificati come pre e post condizioni che specificano la semantica dell'operazione

OCL (Object Constraint Language) è un linguaggio formale usato in UML per esprimere vincoli

- in teoria, pre-condizione e post-condizione delle operazioni possono essere espresse in OCL

Contratti delle operazioni e UP

- possono essere usati per descrivere operazioni di sistema
 - nel Modello dei casi d'uso
- non sono richiesti durante l'ideazione
- vengono scritti principalmente durante l'elaborazione – solo operazioni complesse